

VIBHAS NAIR

(336) 615-5360 | nairvibhas17@gmail.com | [linkedin.com/in/vibhasn/](https://www.linkedin.com/in/vibhasn/) | github.com/nairvibhas18

EDUCATION

University of North Carolina at Chapel Hill

Computer Science B.S., Mathematics B.S.

Expected Graduation: Dec 2027

GPA: 3.7

Relevant Coursework: Foundations of Software Engineering, Data Structures and Algorithms, Machine/Deep Learning

Leadership: COMP 301: Foundations of Programming Teaching Assistant (Aug. 2025 - Dec. 2025)

TECHNICAL SKILLS

Languages: TypeScript, JavaScript, Python, Java, Kotlin, SQL, Go, C, C++, HTML, R, Ruby

Developer Tools: Node, GitHub Actions, Vercel, AWS, Docker, Kubernetes, FastAPI, Supabase, MongoDB

Libraries/Frameworks: React, Next.js, GraphQL, Prisma, PostgreSQL, PyTorch, TensorFlow, scikit-learn, NumPy

WORK EXPERIENCE

Amazon

Software Development Intern

June 2026 – August 2026

Seattle, WA

- Shipped an event-driven **AWS** pipeline in **Kotlin** that grades AI oncall agents against human fixes and proposes spec-file edits, improving root-cause accuracy on unseen tickets to **78%** and cutting wrong diagnoses by **~50%**.
- Wrote **277** tests (**Kotest**) with test code exceeding production code, running end-to-end cases through S3 events.
- Hardened an agent reading untrusted ticket text against **prompt injection** with a fail-closed hook (**146** test cases) that vets every tool call, pins least-privilege roles, and confines all writes to one bucket.
- Eliminated all manual deploy steps by provisioning **TypeScript** CDK infrastructure behind a self-updating pipeline gated on security scans and end-to-end tests.

Infosys

Backend Engineering Intern

May 2025 – July 2025

Raleigh, NC

- Reduced manual lookup time by **40%** and cut query latency to **under 3 seconds** by building an agentic **Retrieval Augmented Generation** system that indexed **10,000+** corporate documents using **Pydantic AI**.
- Designed and deployed a production internal chatbot in **Python** and **Docker**, integrating internal APIs so employees could search company databases, look up resources, and navigate policy.

Computer Science for Social Good – Beautiful Together

Full Stack Developer

Sept. 2024 – May 2025

Chapel Hill, NC

- Facilitated adoption outcomes for **2,400+** animals by engineering a weighted preference matching algorithm that dynamically ranks candidates using real-time data scraped via **BeautifulSoup**.
- Built the full-stack application with a 10-member team using **React (Next.js)**, holding code reviews and managing the end-to-end development lifecycle from database design to deployment.
- Automated production deployment on **Vercel** via **GitHub Actions**, and optimized database queries (**SQL**) for scalability and real-time reliability.

PROJECTS AND RESEARCH EXPERIENCE

Hybrid Attribution & Pruning for Transformer Circuits

Publication / Algoverse AI

May 2025 – Dec. 2025

Remote

- Co-first author** of a paper accepted to **NeurIPS 2025** Mechanistic Interpretability (**MechInterp**) and New Perspectives in Graph ML (**NPGML**) Workshops.
- Designed a novel Hybrid Attribution and Pruning (HAP) framework that combines attribution patching with edge pruning to discover faithful LLM circuits **46% faster** than state-of-the-art (SOTA) methods.
- Implemented custom evaluation pipelines in **PyTorch/TensorFlow**, optimizing GPU memory usage on NVIDIA H100 clusters to benchmark performance against SOTA methods (hosted on **Runpod**).

LoSeR: Decomposable High-Frequency Trading Architecture

Graduate Research Project

Sep. 2025 – Dec. 2025

Chapel Hill, NC

- Architected a novel hybrid framework (**LoSeR**) that decomposes time-series into linear and spectral trends, using **Statistical Recurrent Units** to model residuals and outperform **LSTMs** in capturing long-range dependencies.
- Engineered a multi-horizon forecasting engine (predicting 1, 10, 30, 60-minute intervals) that implements dynamic uncertainty-based thresholding (RMSE scaling) to optimize trade execution, demonstrating profitable generalization on high-volatility S&P 500 data.

RosterQL: GraphQL Class-Roster API

Independent Project

Aug. 2026 – Present

Remote

- Built a **GraphQL** API on **Node** backed by **Prisma** and **PostgreSQL**, with **10** typed roster and class-update operations so teachers and families read the same live class data.
- Shipped a **Next.js (React)** client against that schema, keeping p95 roster queries under **200ms** on local Postgres with seeded data for **40** students.